

Tugas Modul Praktikum 3

Nama: Muhammad Akbar Ramadhani

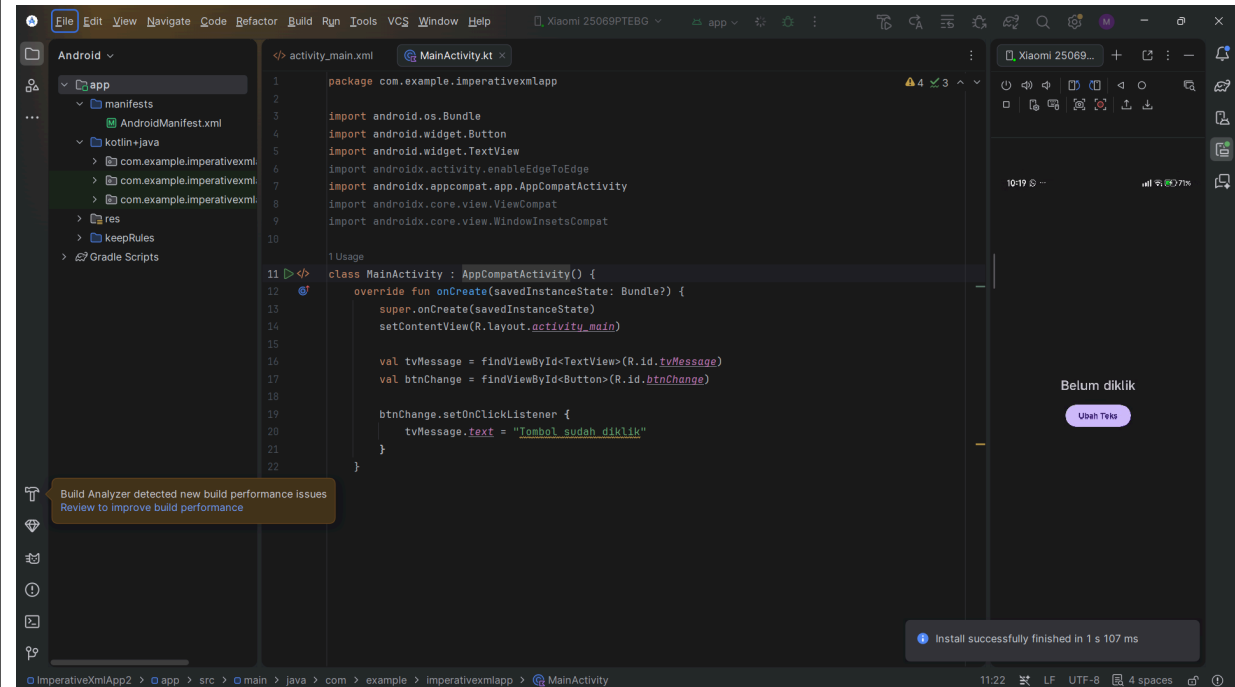
NIM: 245150401111050

Matkul: Pemrograman Aplikasi Mobile - D

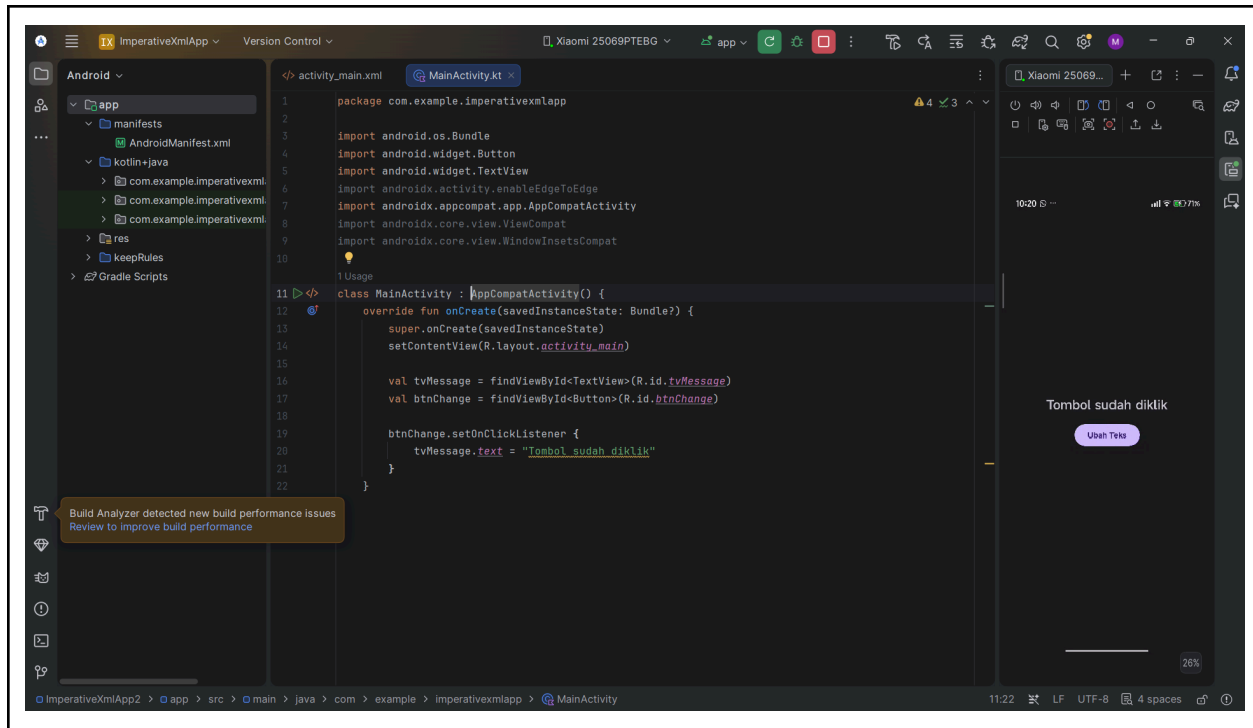
Dosen: Muhammad Aminul Akbar, S.Kom., M.T

Project A - XML/View (Imperatif)

Sebelum Klik:

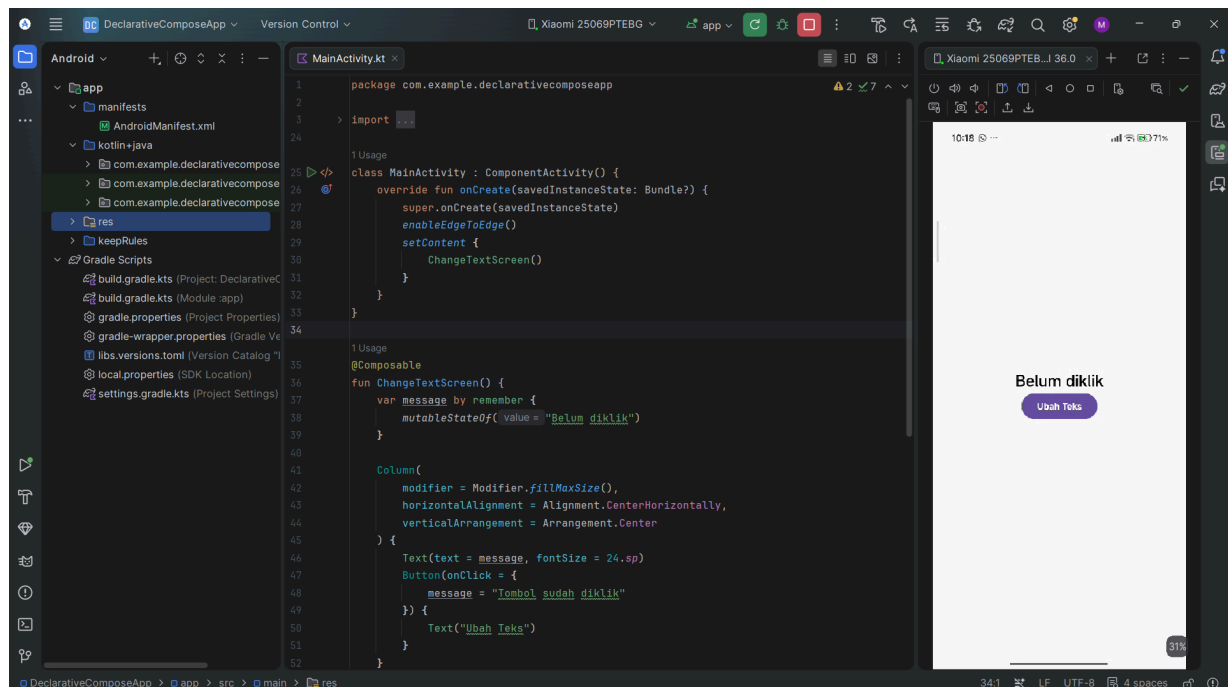


Setelah Klik:

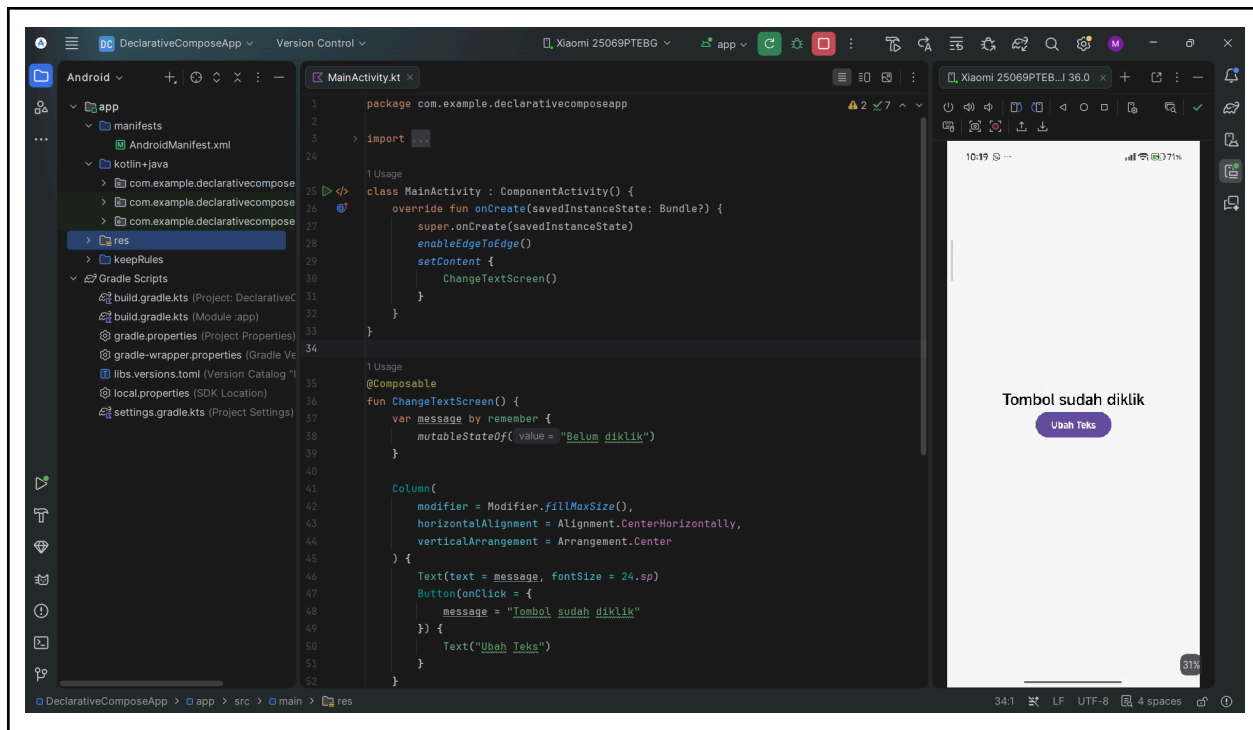


Project B - Jetpack Compose (Deklaratif)

Sebelum Klik:



Setelah Klik:



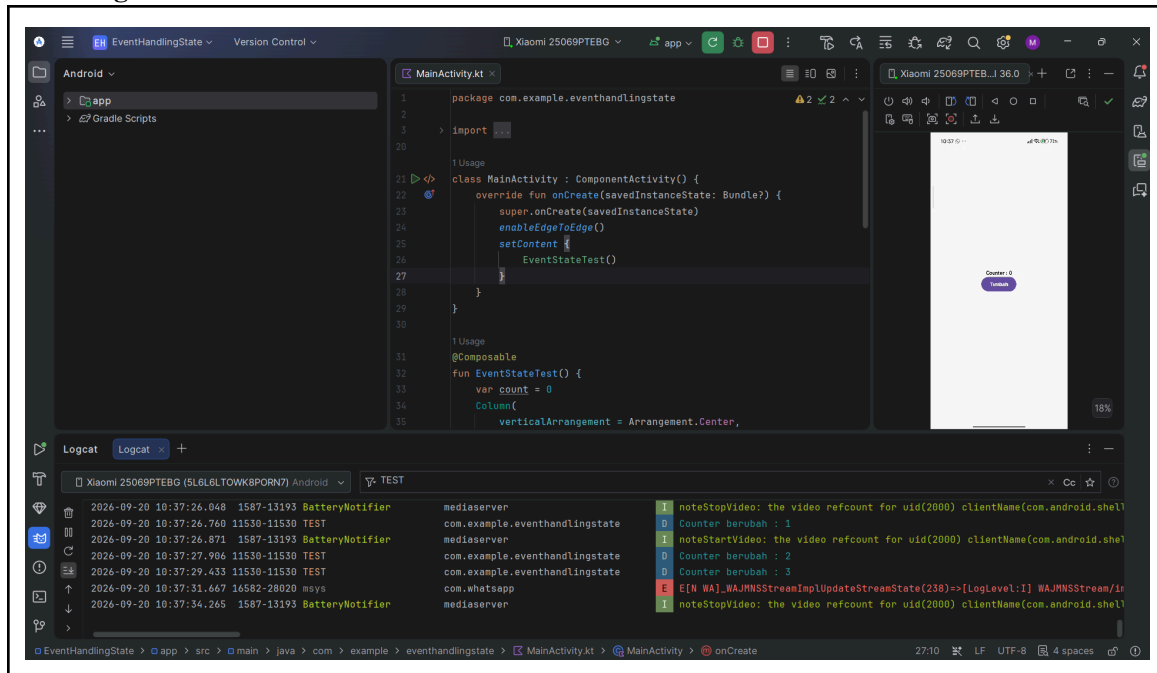
Aktivitas Percobaan

1. Pada project XML/View (Imperatif), objek yang diubah secara langsung adalah komponen View itu sendiri, dalam hal ini properti `text` dari `TextView`. Baris kodenya adalah: `tvMessage.text = "Tombol sudah diklik"`.
2. Pada project Compose (Deklaratif), nilai yang diubah adalah `state` yang menjadi sumber data bagi UI, yaitu variabel `message`. Baris kodenya adalah: `message = "Tombol sudah diklik"`.
3. Jetpack Compose menggunakan pendekatan deklaratif, sehingga kamu tidak mencari dan mengubah objek UI secara langsung. Kamu hanya mendeklarasikan bahwa antarmuka berasal dari suatu `state`, dan Compose akan memperbarui UI secara otomatis saat `state` tersebut berubah.
4. **Alur XML/View:** Saat terjadi interaksi seperti klik (**event**), program akan mencari komponen UI atau **View** yang dituju, lalu program secara langsung mengubah propertinya, yang menghasilkan **perubahan tampilan**.
5. **Alur Compose:** Saat interaksi terjadi (**event**), program hanya memperbarui data atau **state**. Perubahan `state` ini memberitahu Compose untuk mengeksekusi ulang bagian kode UI yang terpengaruh (**recomposition**), yang pada akhirnya menghasilkan **perubahan tampilan**.
6. Jika menggunakan variabel biasa, tampilan teks di layar tidak akan ikut berubah saat tombol diklik. Variabel lokal biasa bisa berubah nilainya di memori, tetapi perubahan tersebut tidak memicu Compose untuk melakukan *recomposition* atau menggambar ulang UI. Agar Compose tahu data berubah, harus menggunakan `state` yang dapat diamati seperti `mutableStateOf()`.
7. Kesimpulannya, cara berpikir imperatif itu seperti memberikan instruksi manual satu per satu: "cari komponen A, lalu ubah nilainya menjadi B". Sedangkan cara deklaratif lebih fokus pada hasil akhir: "tampilan harus selalu mencerminkan data saat ini"; cukup ubah datanya, dan antarmuka akan mengurus perubahannya sendiri.

Event Handling State

Aktivitas Percobaan

1. Hasil Logcat:



2. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?

- LogCat akan mengeluarkan pesan seperti "Counter berubah 1", "Counter berubah 2", dan seterusnya secara berurutan.

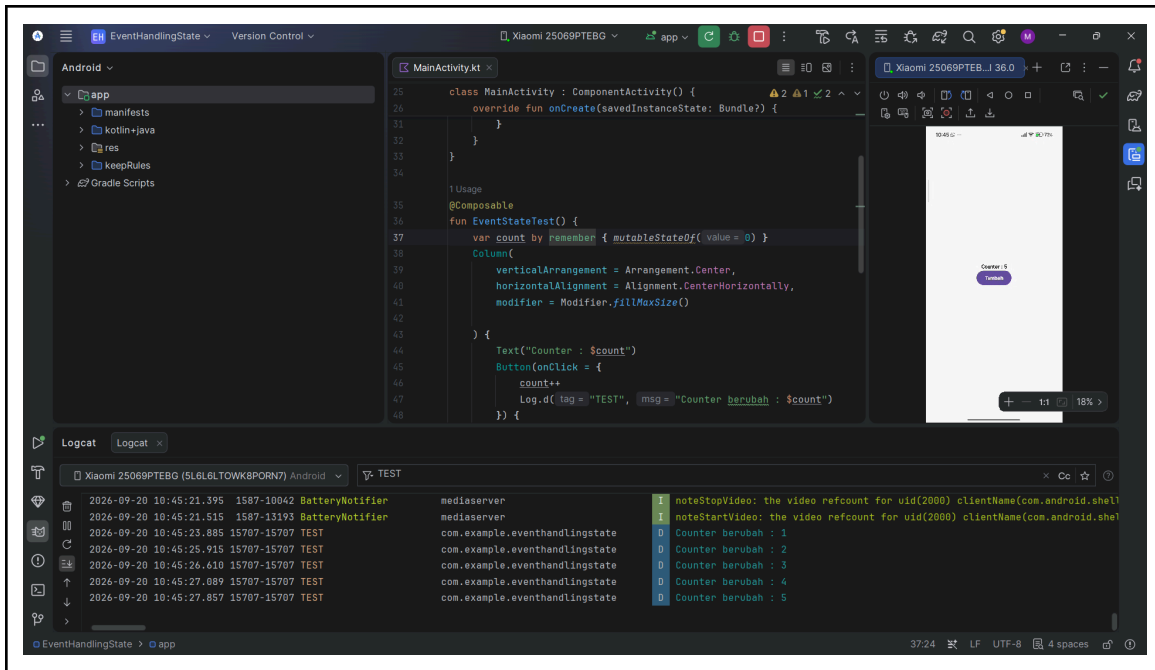
3. Apa yang terjadi pada paparan aplikasi setiap kali tombol "Tambah" diklik?

- Paparan teks pada aplikasi tetap sebagai "Counter: 0" dan tidak berubah sama sekali.

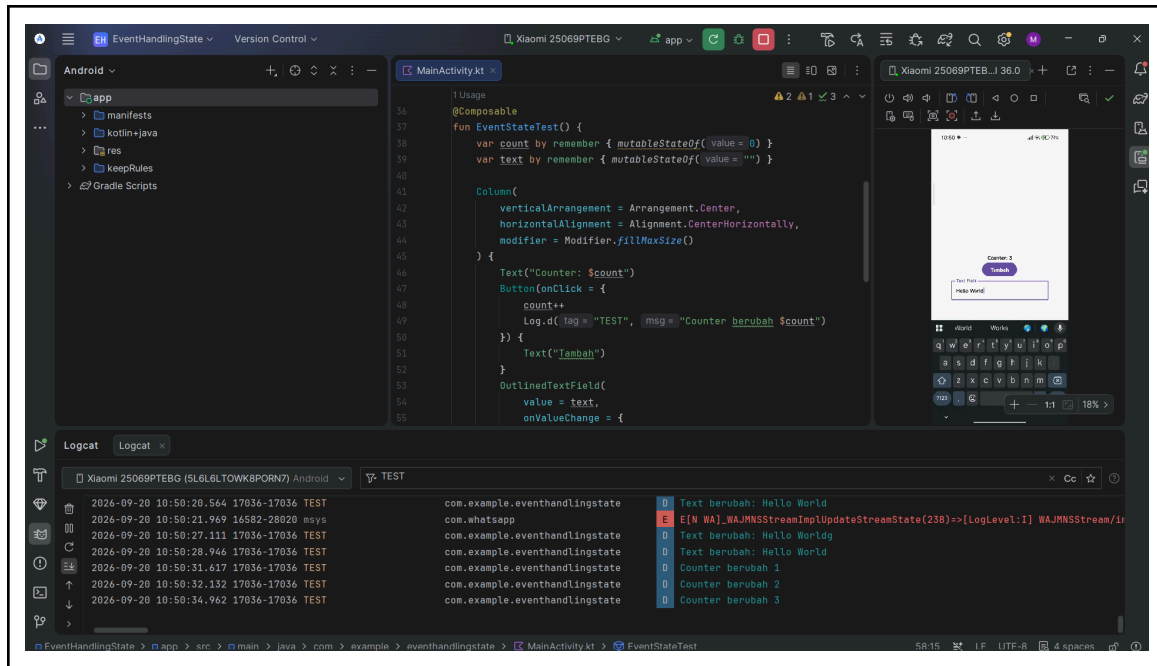
4. Apa yang menyebabkan hal tersebut terjadi?

- Ini berlaku kerana pembolehubah `count` yang digunakan hanyalah pembolehubah biasa. Walaupun nilainya bertambah di latar belakang (seperti yang dilihat di LogCat), Jetpack Compose tidak menyedari perubahan tersebut. Perubahan pada pembolehubah biasa tidak secara automatik meminta Compose untuk mengemas kini atau melukis semula antaramuka pengguna (*recomposition*).

5. Hasil Logcat setelah diubah menjadi State:



6. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?
 - Pesan di LogCat tetap muncul seperti sebelumnya, yaitu "Counter berubah 1", "Counter berubah 2", dan seterusnya.
7. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?
 - Berbeda dengan sebelumnya, kali ini angka pada tampilan layar ("Counter: 0") ikut berubah dan bertambah secara *real-time* setiap kali tombol diklik.
8. Apa yang menyebabkan hal tersebut terjadi dan berbeda dengan percobaan sebelumnya?
 - Kali ini kita menggunakan *state* melalui fungsi `mutableStateOf()`. Jetpack Compose memantau *state* ini, sehingga ketika nilainya bertambah (saat tombol diklik), Compose otomatis menyadari perubahan data tersebut dan menggambar ulang bagian UI yang bergantung pada data tersebut.
9. Apa yang dimaksud recomposition pada Compose?
 - Recomposition adalah proses di mana Compose memperbarui bagian UI dengan menjalankan ulang fungsi *composable* ketika *state* yang menjadi sumber datanya mengalami perubahan.
10. Hasil Logcat setelah ditambah state baru:



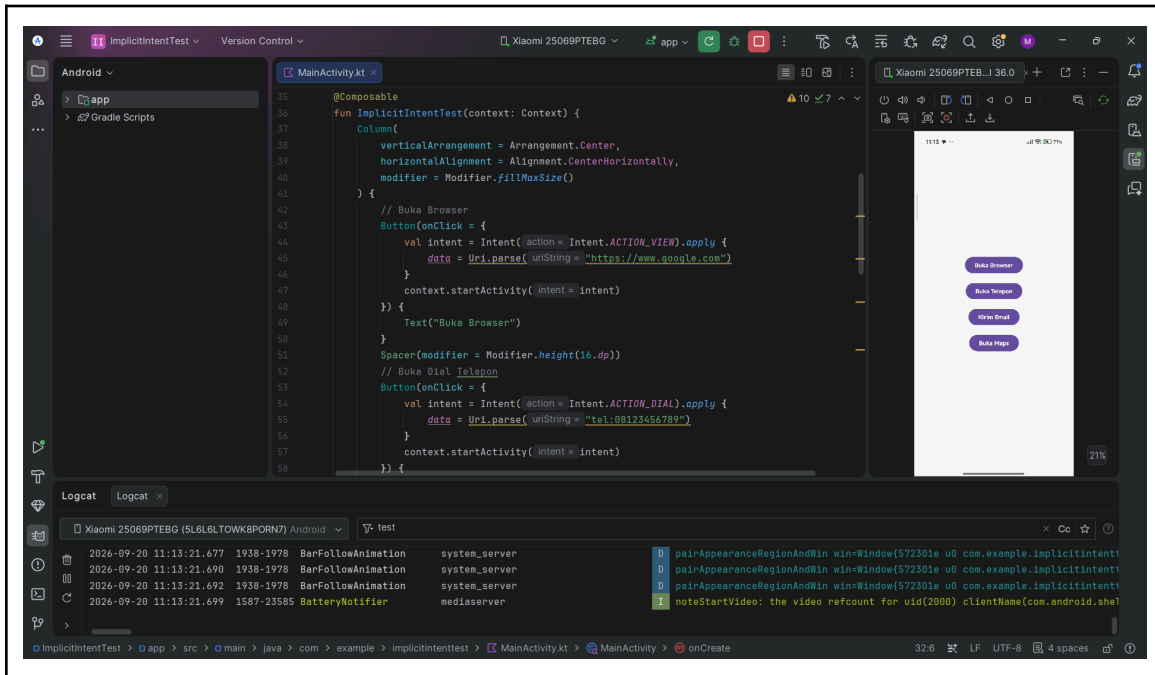
11. Apa yang muncul di LogCat setiap kali menginputkan huruf di Text Field?
 - Di LogCat akan tercetak pesan "Text berubah: " diikuti dengan teks yang kamu ketik (berasal dari perintah `Log.d("TEST", "Text berubah: $it")`).
12. Apa yang terjadi pada tampilan aplikasi setiap kali menginputkan huruf di Text Field?
 - Teks yang kamu ketikkan akan langsung muncul dan diperbarui di dalam kotak input pada layar aplikasi.
13. Apa yang menyebabkan hal tersebut terjadi?
 - Setiap kali kamu mengetik, hal tersebut memicu *event* `onValueChanged` yang kemudian memperbarui nilai *state* `text` (`text = it`). Karena *state* tersebut berubah, Compose melakukan *recomposition* dan menggambar ulang komponen `OutlinedTextField` dengan nilai terbaru.
14. Sebutkan state apa saja yang ada pada program yang terakhir dibuat?
 - Ada dua *state* yang digunakan: `count` (untuk menyimpan angka) dan `text` (untuk menyimpan string teks).
15. Sebutkan event apa saja yang ada pada program yang terakhir dibuat?
 - Ada dua *event*: `onClick` pada tombol Button dan `onValueChanged` pada OutlinedTextField.
16. Kapan event-event tersebut dijalankan?
 - `onClick` dijalankan secara spesifik saat pengguna mengklik atau menekan tombol "Tambah", sedangkan `onValueChanged` dijalankan setiap kali ada perubahan input (mengetik atau menghapus huruf) di dalam kotak teks.

Implisit Intent

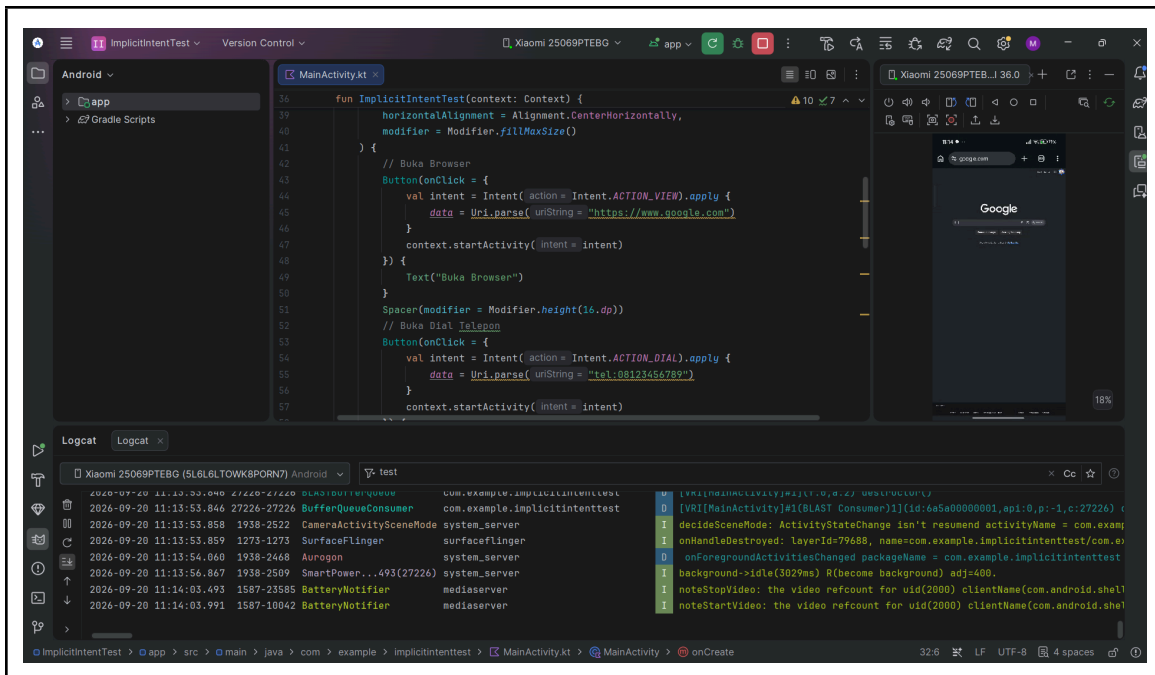
Aktivitas Percobaan

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Lakukan pengujian pada setiap tombol:

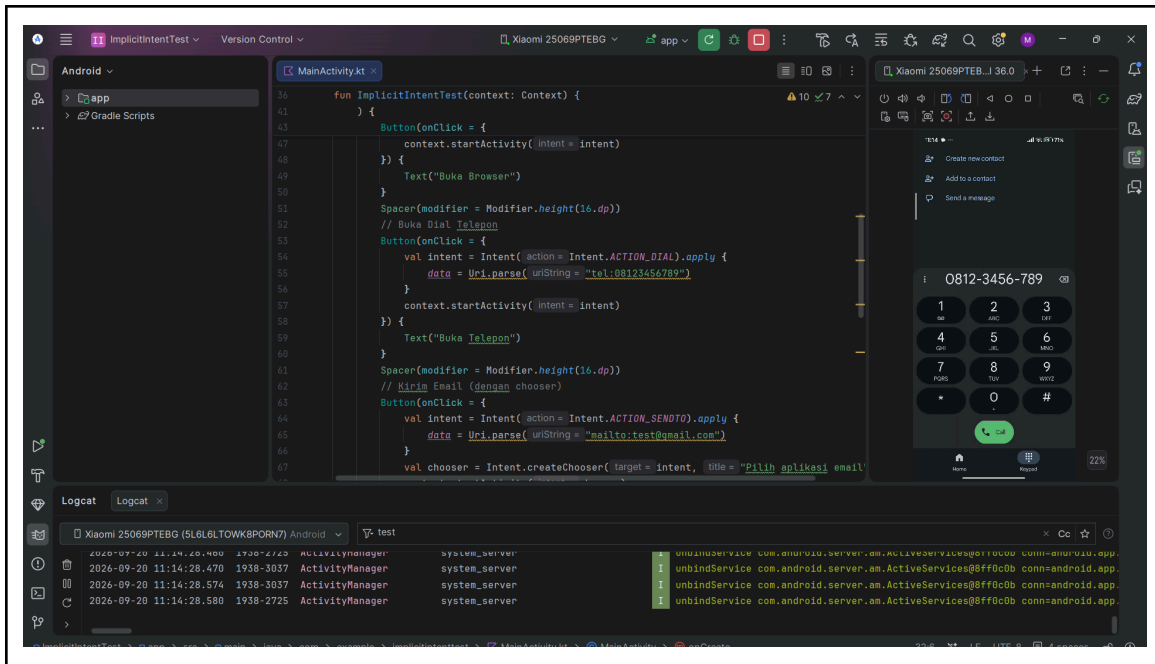
2. Hasil Run Kode:



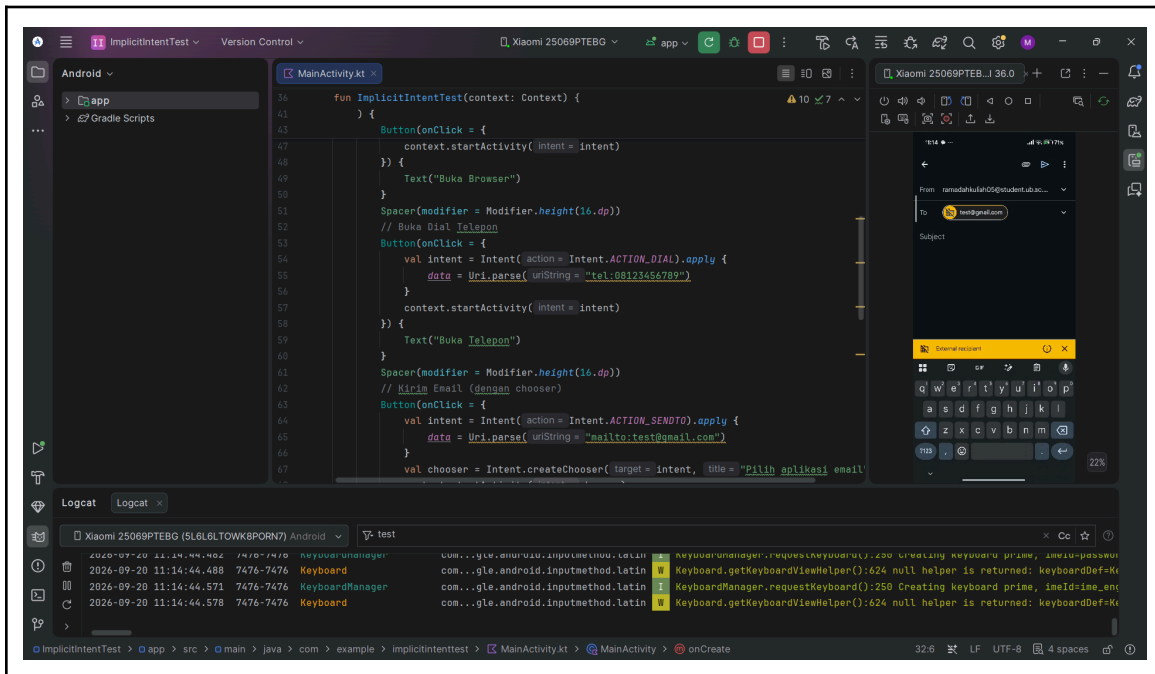
3. Hasil Klik Browser:



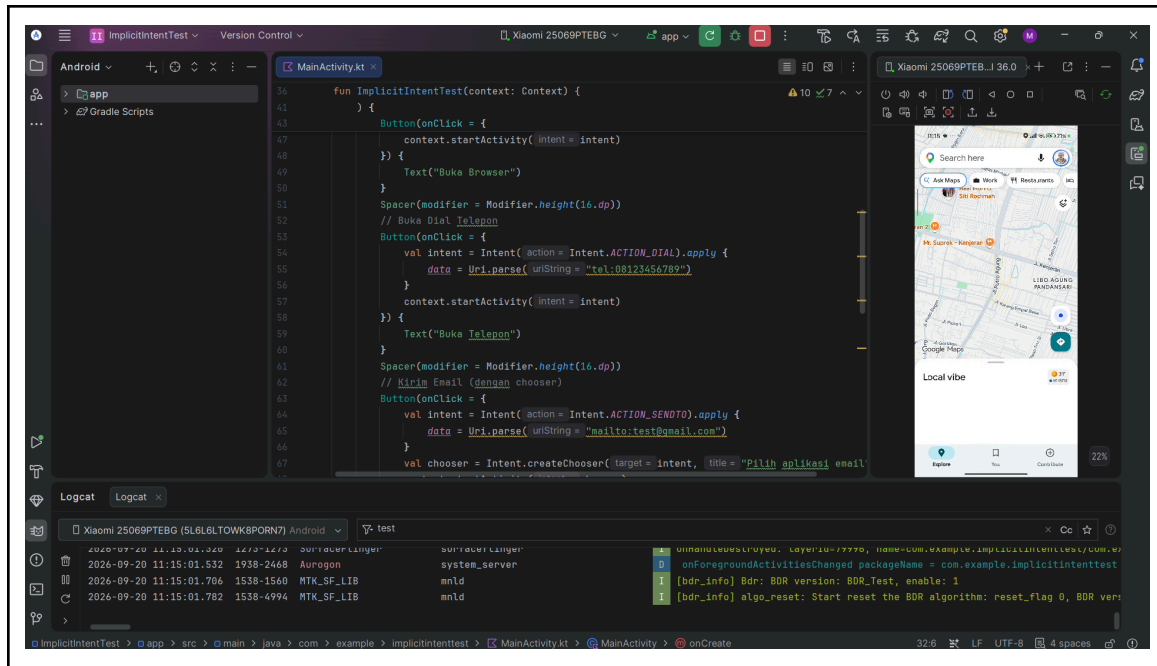
4. Hasil Klik Telepon:



5. Hasil Klik Email:



6. Hasil Klik Maps:



7. Apa perbedaan fungsi dari masing-masing action intent tersebut?

- **Intent.ACTION_VIEW**: Digunakan untuk menampilkan suatu informasi kepada pengguna berdasarkan tipe datanya, misalnya membuka halaman web di browser atau menampilkan koordinat di aplikasi peta.
- **Intent.ACTION_DIAL**: Digunakan secara khusus untuk membuka aplikasi telepon (dialer) dengan nomor yang sudah otomatis terisi di layar, siap untuk dihubungi.
- **Intent.ACTION_SENDTO**: Digunakan untuk mengirim pesan kepada seseorang secara spesifik, yang pada kode ini dimanfaatkan untuk membuka aplikasi pengirim email.

8. Apa fungsi data pada intent?

- Fungsi **data** (yang diformat menggunakan **Uri.parse**) adalah memberikan informasi spesifik mengenai target yang harus diproses oleh Action tersebut. Informasi ini bisa berupa URL *website* tujuan, nomor telepon target, alamat email penerima, atau titik kordinat garis lintang dan bujur peta.

9. Apa hubungan Action dan Data pada pilihan aplikasi yang dimunculkan oleh sistem Android?

- Pada *Implicit Intent*, kita tidak menentukan aplikasi mana yang akan dibuka. Sistem Android menganalisis kombinasi **Action** (apa yang ingin dilakukan) dan **Data** (tipe formatnya seperti **https:**, **tel:**, **mailto:**, atau **geo:**) untuk menyaring aplikasi apa saja di dalam *smartphone* yang mendukung operasi tersebut. Jika Android menemukan lebih dari satu aplikasi yang cocok (misalnya kamu menginstal Gmail dan Outlook), sistem akan memunculkan kotak dialog (*chooser*) agar pengguna bisa memilih sendiri aplikasi mana yang mau dipakai.